Intelligence Never Grants Authority:

A STAMP-Compliant Control Structure for Autonomous Agents

Jonathan R. M. Kershaw

Originator, Agent Responsibility Engineering

srexai.dev | github.com/srex-dev

2026

Abstract

Autonomous agent systems are in production. They act across consequential decision boundaries in regulated environments. The governance frameworks built to manage them are not working. Prompt engineering constrains inputs. Output filtering catches some outputs. Human checkpoints slow down the most dangerous decisions. None of these is a control structure. None of them would satisfy a safety engineer.

This paper connects two things that have not been connected in the literature. The first is Nancy Leveson's Systems-Theoretic Accident Model and Processes -- fifty years of safety engineering research that explains, with formal precision, why complex systems fail and what it takes to prevent them from failing. The second is Agent Responsibility Engineering -- a governance discipline built from scratch to govern autonomous agents in regulated production environments.

The connection is not approximate. STAMP requires a controller, a controlled process, control actions, feedback, a process model, safety constraints, and a closed control loop. ARE provides all of them. The formal correspondence across fifteen mappings is complete and bidirectional within a bounded scope defined in Section 4.1. Every STAMP concept has a named ARE implementation. Two ARE-native extensions -- evidence-based control and predictive control via LLM-MPC -- extend the mapping beyond classical STAMP.

Three components extend STAMP to cover properties that do not exist in physical systems. The Agent Passport makes identity -- which STAMP assumes but physical systems fix by architecture -- explicit and cryptographically enforceable. The Operations Ledger provides trust-minimized feedback: tampering is detectable by hashchain, not assumed impossible. The Governance

Coprocessor physically interposes the control structure between agent intent and execution, satisfying Leveson's requirement that control actions be applied, not recommended.

The architecture has been validated in production in a regulated financial services environment. The reference implementation is available at github.com/srex-dev/guardian-agent.

STAMP was always the right frame for this problem. ARE is the minimum viable control structure that frame requires. This paper closes the connection.

1. The Problem

Agents are in production. Not demos. Not pilots. Production systems making consequential decisions in regulated environments -- financial services, healthcare, critical infrastructure. They reason, plan, and act. They call APIs, read records, initiate transactions, and generate outputs that affect real people under real regulatory obligations.

The governance approaches that exist were not designed for this. They were designed for a simpler version of the problem: a model that takes input and produces output, with a human reviewing the output before anything happens. That version of the problem is already obsolete.

Prompt engineering constrains what the agent is told to do. It does not constrain what the agent can do. Output filtering catches some classes of bad outputs after the agent has acted. Human checkpoints insert review at defined decision points, but agents act faster than checkpoints can follow and the checkpoints are defined in advance of failure modes that will actually matter. None of these approaches provides what safety engineering has known for fifty years is required: a control structure.

A control structure is not a rule set. It is not a monitoring system. It is not a policy document.

A control structure is the complete system of controllers, controlled processes, control actions, and feedback mechanisms whose interaction produces safety as a property of the architecture. Safety engineering learned this by studying accidents. Accidents in complex systems do not happen because a component fails. They happen because the control structure failed -- because the controller was operating on a process model that had drifted from reality and issued actions that were wrong, or missing, or too late.

Agent governance has not been analyzed through this lens. The failure modes that STAMP predicts for inadequate control structures are exactly the failure modes that ungoverned agent systems exhibit. The theoretical foundation has existed for fifty years. It has not been applied here.

Agent Responsibility Engineering is the application. This paper is the formal account of the connection.

2. STAMP: What It Says and Why It Applies

Nancy Leveson developed Systems-Theoretic Accident Model and Processes over decades of research into why complex sociotechnical systems fail. The core argument is a departure from how most engineers think about accidents.

The traditional model says accidents happen because components fail. Find the broken component. Fix it. The model that replaced it in safety-critical engineering says accidents happen because of inadequate control. The system was not broken. The control structure failed to keep it safe.

> *Safety is a control problem. Accidents result from a lack of appropriate constraints on the behavior of the system's components and their interactions. -- Leveson, Engineering a Safer World, 2011*

This reframing has consequences. If accidents are control failures, then safety engineering is the engineering of control structures. The question is not what broke but what the control structure failed to prevent, and why.

2.1 The Core Model

STAMP's analytical vocabulary is precise and small. A **loss** is an unacceptable outcome the system exists to prevent. A **hazard** is a system state that, given worst-case environmental conditions, leads to a loss. A **safety constraint** is a behavioral boundary whose enforcement prevents hazard state entry.

The **control structure** is the complete system responsible for enforcing safety constraints. It has controllers that issue control actions, controlled processes that respond to them, and feedback mechanisms that return information about the controlled process state to the controller. Safety is a property of this structure, not of individual components.

The **process model** is the controller's internal representation of the controlled process state. The controller does not act on reality. It acts on its model of reality. If the model is accurate, control actions will be appropriate. If it has drifted -- stale, incomplete, corrupted, or manipulated -- control actions will be wrong. Accidents follow.

Keep the process model accurate. Everything else is in service of that.

2.2 The Feedback Requirement

The process model stays accurate through feedback. The controller issues a control action. The controlled process responds. Feedback returns information about that response to the controller, updating the process model before the next control action. This is a closed control loop.

Open-loop control -- action without feedback -- cannot maintain process model accuracy over time. The model drifts. Actions become inappropriate. STAMP predicts this failure mode. It is not an edge case. It is the expected outcome of open-loop governance in a changing system.

Most agent governance frameworks are open-loop. They constrain or monitor agent behavior but provide no mechanism for the governance system to update its model of what the agent is doing based on observed outcomes. The governance system issues constraints and then loses contact with the controlled process. STAMP predicts exactly what happens next.

2.3 Why STAMP Has Not Been Applied Here

STAMP was developed for physical controlled processes. Its implicit assumptions reflect that origin. The controller is distinct from the controlled process. The controlled process has a fixed identity determined by physical architecture. Feedback is generated by physical sensors the controlled process cannot manipulate.

Autonomous agent systems violate all three assumptions. An agent can act as both controller and controlled process in nested workflows. Agent identity is not fixed -- it must be constructed and verified at runtime. An agent system generates its own records and can, without integrity controls, manipulate them.

Prior STAMP applications to software-intensive systems exist -- primarily in aerospace and safety-critical embedded contexts. Applications to AI systems have focused on model-level safety analysis: examining individual inference pipelines, analyzing failure modes of specific components. No prior work uses STAMP as the theoretical foundation for the governance layer that sits above those components and manages their actions in production.

The violations of STAMP's physical assumptions do not invalidate the framework. They require its extension. The three extensions ARE provides are precisely targeted: the Passport for identity, the Ledger for manipulation-resistant feedback, the Coprocessor for physical interposition. Each addresses exactly one place where agent systems break STAMP's assumptions, and nothing else.

3. Agent Responsibility Engineering

Agent Responsibility Engineering is a governance discipline for autonomous agent systems in high-consequence environments. It was built to solve the production problem, not the theoretical one. The theoretical account came after.

The governing principle: intelligence never grants authority. Authority is earned through verifiable policy enforcement.

This is not a policy preference. It is an engineering requirement. An agent that is highly capable is not therefore trustworthy. Capability and authority are different properties. ARE treats them as different properties and provides a mechanism for granting authority that is independent of capability assessment.

ARE is specified across four layers. Each layer corresponds to a distinct set of control structure responsibilities. The layers are not organizational. They are functional.

3.1 Foundational Layer: Identity and Authority

The Foundational layer answers a question STAMP assumes is already answered: who is the controlled entity, and what is it authorized to do?

Every agent operating under ARE governance holds an **Agent Passport**: a cryptographic identity artifact issued at instantiation. The Passport encodes the agent's unique identifier, its delegated authority, its behavioral constraints, its operational scope, its chain of delegation from the issuing

principal, and its expiry condition. It is signed by the issuing authority and verified at every policy evaluation.

Without the Passport, the control structure cannot attribute actions to specific agents, cannot enforce per-agent authority boundaries, and cannot maintain an agent-specific process model. The Passport is what makes agents legible to the governance system. Legibility is a precondition for control.

The **Delegation Model** governs how authority propagates through multi-agent workflows. A parent agent delegating to a child cannot grant authority it does not hold. The Passport encodes the delegation chain explicitly. The coprocessor enforces delegation boundaries at verdict time. Authority cannot be self-extended. The chain is verifiable and recorded.

3.2 Operational Layer: Real-Time Enforcement

The Operational layer is where governance becomes physical. The **Governance Coprocessor** sits between agent intent and execution. Every agent action must pass through it. No exceptions.

When an agent proposes an action, the coprocessor intercepts it. It verifies the Passport. It queries the Operations Ledger for relevant process model state. It evaluates the proposed action against current policy. It issues a verdict: permit, modify, or block. The action proceeds only on permit or after modification. Block stops execution entirely and records the attempt.

This is a STAMP controller in the precise technical sense. It maintains a process model. It issues control actions. It is physically interposed in the causal chain between the controlled process and the state change it would produce.

A governance system that can be bypassed is not a controller. It is a recommendation.

The coprocessor closes that gap architecturally. Guardian-Agent is the open-source reference implementation -- OPA-based policy evaluation, Passport verification, Ledger integration. Available at github.com/srex-dev/guardian-agent.

3.3 Epistemological Layer: Process Model and Learning

The Epistemological layer maintains the process model and drives learning from observed behavior. The **Operations Ledger** is the authoritative record of system state.

Every agent action, every policy verdict, every state transition, every control intervention is recorded as an immutable, append-only hashchain event. The Ledger is the system's memory. It is also the feedback mechanism that keeps the controller's process model current and verifiable.

The hashchain is a safety property, not an audit convenience. Each event is cryptographically bound to its predecessor. A falsified record breaks the chain. The manipulation is detectable. The Ledger is trust-minimized: it does not assume the agent is honest. It makes dishonesty visible.

3.4 Posture Layer: Control Structure Integrity

The Posture layer manages the control structure itself. It monitors the governance system for degradation. It manages promotion gates -- the criteria governing when an agent's authority can be expanded. It handles circuit-breaking when conditions exceed what the control structure can safely govern.

Posture is the control structure's self-model. Most governance frameworks have no analog. They govern agents but have no mechanism for assessing whether the governance system itself is functioning correctly. Posture is that mechanism.

3.5 SDAVL: The Closed Control Loop

SDAVL is ARE's closed control loop. It executes on every agent action.

Sense: the coprocessor receives an action request and queries the current process model from the Ledger -- the verified behavioral history plus active Passport registry. **Decide**: a policy verdict is issued with a structured evidence bundle encoding the evidentiary basis; LLM-MPC forward simulation evaluates the proposed action against the predicted trajectory across a planning horizon, blocking or modifying actions that lead toward hazard states before they materialize. **Act**: the verdict is enforced. **Verify**: the execution outcome and evidence bundle are recorded to the Ledger as a hashchain event. **Learn**: the process model is updated via hypothesis validation -- competing hypotheses about the event are evaluated against Ledger evidence, and the model is updated based on the hypothesis that survives falsification, not the raw event.

The loop closes. The next action request encounters an updated process model that reflects what actually happened. Not periodic. Continuous. Every action. Every time.

4. Formal Correspondence: STAMP and ARE

Table 1 presents the complete formal mapping between STAMP's theoretical concepts and their ARE implementations. Every STAMP concept has a named ARE artifact. The bidirectionality is the paper's central theoretical claim: every STAMP concept maps to an ARE component, and every ARE component addresses a STAMP requirement.

Table 1. STAMP / ARE Complete Formal Mapping

STAMP Concept	Leveson Definition	ARE Implementation
Loss	Unacceptable outcome the system must prevent	Compliance violation, unauthorized data access, erroneous financial decision, audit failure. ARE extends the loss taxonomy to include regulatory and fiduciary harm classes specific to SR 11-7, GLBA, ECOA, FCRA.
Hazard	System state leading to loss under worst-case conditions	Ungoverned agent action: any action that executes without a verified policy verdict, attributable identity, or evidence bundle. Secondary hazards: delegated authority exceeding granted scope, process model drift, coprocessor bypass.
Safety Constraint	Behavioral boundary preventing hazard state entry	The ARE Tenets. Each tenet defines a boundary whose violation is hazard state entry. Not principles. Not guidelines. Constraints enforced by a coprocessor physically interposed between intent and execution.
Controller	Component issuing control actions based on	Governance Coprocessor. Receives every agent action request. Verifies Passport. Evaluates policy. Queries Ledger

STAMP Concept	Leveson Definition	ARE Implementation
	process model	state. Issues verdict: permit, modify, or block. Physically in the causal chain. Nothing executes without passing through it.
Controlled Process	System whose behavior the controller manages	Agent or agent collective. Partially opaque -- internal reasoning not directly observable. Passport and Ledger make external behavior legible to the control structure. See Section 4.1 for observability boundaries.
Control Actions	Commands issued by controller to controlled process	Policy verdicts: permit, modify, block -- each issued with a structured evidence bundle encoding the evidentiary basis for the verdict. A verdict without an evidence bundle is an assertion. A verdict with an evidence bundle is a verifiable control action. A missing verdict is a missing control action. Hazard state entry by definition.
Feedback	Information returned from controlled process to controller	Operations Ledger. Every agent action, policy verdict, state transition, and control intervention recorded as an immutable hashchain event. Continuous. Append-only. Trust-minimized: tampering is detectable, not assumed impossible.
Process Model	Controller's internal representation of controlled process state	Current Ledger state plus active Passport registry, interpreted through hypothesis validation. Ledger provides verified behavioral history. Hypothesis validation distinguishes genuine behavioral drift from variance before process model updates trigger governance responses. Formally verifiable within the boundaries defined in Section 4.1.
Control Structure	Complete system of controllers, processes, actions, and feedback	ARE architecture across four layers: Foundational (identity, authority), Operational (real-time enforcement), Epistemological (process model, learning, hypothesis validation), Posture (control structure integrity, adaptation).
Inadequate Control	Missing, wrong, or mistimed control actions	Ungoverned action (missing verdict). Unverified verdict (missing evidence bundle). Incorrect verdict (wrong action). Passport expiry undetected (stale model). Ledger integrity failure (feedback degradation). Coprocessor bypass (structure failure). Each has a named prevention mechanism.
Controller Authority	Legitimate scope of controller command over controlled process	Delegation Model. Every Passport encodes delegated authority at issuance. Coprocessor enforces authority boundaries at verdict time. Authority cannot be self-extended. STAMP assumes this. ARE enforces it cryptographically.
Context / Environment	Operating conditions affecting control structure	Regulated financial services environment. SR 11-7, GLBA, ECOA, FCRA. Encoded as policy in the coprocessor. Compliance is inside the control loop, not external audit.
Closed Control Loop	Continuous feedback maintaining process model accuracy	SDAVL: Sense (action request received, process model queried), Decide (policy verdict with evidence bundle issued; LLM-MPC forward simulation evaluates trajectory safety across planning horizon), Act (verdict enforced), Verify (outcome and evidence bundle recorded to Ledger), Learn (process model updated via hypothesis validation). Executes on every agent action. Continuous, not periodic.

STAMP Concept	Leveson Definition	ARE Implementation
Evidence-Based Control	ARE extension: verifiable evidentiary basis for all control actions	Every policy verdict is accompanied by a structured evidence bundle classifying the evidentiary basis. Evidence bundle classification makes control actions auditable, not just enforceable. The bundle is recorded to the Ledger as part of the verdict event. A control structure that cannot show its reasoning cannot be audited. Evidence-based control closes that gap.
Predictive Control	ARE extension: forward-looking control via LLM-MPC	LLM-MPC applies classical Model Predictive Control to agent decision-making. At the Decide step, rather than evaluating only the current proposed action against current state, LLM-MPC simulates forward: given the current process model, what are the likely future states if this action is permitted? If the predicted trajectory converges on a hazard state, the coprocessor blocks or modifies now -- before the hazard materializes. Transforms the control structure from reactive to predictive.

4.1 Scope and Boundaries of the Correspondence

The claim that no gap exists requires precise scoping. The mapping is complete within the following boundaries. Stating them explicitly is not a weakness. It is what makes the claims that fall within the boundaries defensible.

What the mapping covers: the enforcement of behavioral constraints on agent actions at the point of execution. Every action an agent proposes passes through the coprocessor. Every coprocessor decision is recorded to the Ledger. Every Ledger event updates the process model. The control loop is closed and continuous for all actions within the governed system boundary.

What the mapping does not claim: the mapping does not assert control over agent internal reasoning -- the cognitive process by which an agent arrives at a proposed action. An agent's internal state is not directly observable. ARE governs what agents do, not what they think. This is the correct scope for a control structure. STAMP governs the behavior of controlled processes, not their internal mechanics.

Partial observability: ARE's answer to the partial opacity of agent reasoning is adequacy, not completeness. The Passport establishes what the agent is authorized to do. The Ledger records what it actually does. The gap between internal reasoning and external action is where residual risk lives. That gap is bounded by the enforcement layer: a controlled process that cannot execute an action without a policy verdict cannot act on unsafe internal reasoning without detection or prevention at the execution boundary.

Emergent multi-agent behavior: the mapping addresses individual agent governance and first-order delegation chains. Emergent behavior across large agent collectives -- properties that arise from interaction rather than individual action -- is outside the current scope. Section 5.2 addresses the multi-agent recursion problem within the delegation model. Emergent collective behavior is identified as future work in Section 11.1.

Policy correctness versus enforcement correctness: ARE provides enforcement correctness -- the coprocessor enforces whatever policy it is given. Policy correctness -- whether the policy

itself is right -- is a separate concern. ARE does not solve policy design. It guarantees that the policy that exists is enforced without exception.

4.2 Three Extensions to STAMP

Three rows in Table 1 represent extensions of STAMP beyond its original physical-system formulation. These are the places where agent systems break STAMP's implicit assumptions and where ARE provides the solution.

The **Controlled Process** row addresses partial opacity. STAMP assumes the controlled process is observable through sensors. An agent's internal reasoning is not. The Passport provides the identity anchor. The Ledger provides verified behavioral history. Together they make the controlled process legible at the boundary that matters: the action boundary.

The **Feedback** row addresses manipulation resistance. Physical sensors cannot falsify their own readings without physical intervention that is itself observable. An agent system can generate records of its own actions. The hashchain makes tampering detectable. This is trust-minimized feedback, not trusted feedback. The distinction matters.

The **Controller Authority** row makes explicit what STAMP assumes. In physical systems, authority is fixed by wiring. In agent systems, authority must be cryptographically bound at issuance and verified at enforcement time. The Passport and Delegation Model satisfy this. STAMP assumed it. ARE builds it.

4a. ARE-Native Extensions to STAMP

The fifteen-row mapping in Table 1 contains thirteen direct STAMP correspondences and two extensions that go beyond classical STAMP. These extensions are not gaps in the correspondence. They are places where the agent governance context requires capabilities that STAMP does not address because they were not required for physical systems. They represent the original theoretical contribution of ARE beyond the synthesis work.

4a.1 Evidence-Based Control

STAMP requires that control actions be appropriate given the current process model. It does not specify how appropriateness is demonstrated or verified after the fact. In physical systems, this is not a concern: a valve either opens or it does not. The control action is its own evidence.

In autonomous agent systems, the reasoning behind a policy verdict is not self-evident. A coprocessor that blocks an action has made a judgment. That judgment must be auditable -- not just in the sense that it is recorded, but in the sense that the evidentiary basis for the judgment is preserved and inspectable.

Evidence bundles solve this. Every policy verdict issued by the coprocessor is accompanied by a structured evidence bundle: a classified record of what the coprocessor observed, what policy it applied, what evidence it weighed, and what it concluded. The bundle is recorded to the Ledger as part of the verdict event. It is cryptographically bound to the verdict by the hashchain.

A minimal evidence bundle contains: the policy clause or tenet triggered, the Passport scope delta between what the agent requested and what it is authorized to do, any prior violations in the agent's Ledger history relevant to the verdict, a classification of the evidence type (scope violation, behavioral drift, delegation boundary breach, policy gap), and a confidence classification for the verdict. This structure is sufficient to reconstruct the reasoning behind any verdict from the Ledger alone, without access to the coprocessor's internal state at the time of evaluation.

Evidence-based control makes two things true that are not true of enforcement alone. First, verdicts are auditable: a regulator, auditor, or incident reviewer can reconstruct not just what verdict was issued but why. Second, verdict patterns are analyzable: systematic review of evidence bundles across the Ledger reveals whether the coprocessor is applying policy consistently and whether the policy itself is producing the intended control effects.

A control structure that can enforce but cannot show its reasoning is a control structure that cannot be trusted with consequential decisions. Evidence bundles close that gap. They are what makes ARE a governance system rather than an enforcement mechanism.

4a.2 Hypothesis Validation and Behavioral Drift Detection

STAMP's Learn step -- updating the process model based on observed outcomes -- is straightforward for physical systems. A sensor reports a reading. The reading updates the model. The update is unambiguous.

For autonomous agent systems, process model updates are not unambiguous. An agent that exceeds its Passport scope once may be drifting. It may also have encountered an unusual edge case. An agent whose response latency increases may be degrading. It may also be processing a legitimately complex request. Naive process model updates -- treating every anomaly as drift -- produce false positives that degrade the governance system's utility. Ignoring anomalies produces false negatives that allow genuine drift to go undetected.

Hypothesis validation is the mechanism by which the Epistemological layer distinguishes signal from noise in the behavioral record. When the Ledger records an anomalous event, the hypothesis engine generates competing hypotheses about its cause -- genuine drift, edge case, environmental factor, policy gap -- and evaluates each against the available evidence in the Ledger. The process model is updated based on the validated hypothesis, not the raw event.

This is the application of deterministic root cause analysis to agent governance. The hypothesis engine does not guess. It generates falsifiable hypotheses, evaluates them against evidence, and updates the process model with the hypothesis that survives falsification. The approach is derived from THE Hypothesis Engine -- a parallel hypothesis generation and falsification system developed as part of the ARE reference architecture.

Hypothesis validation makes the Learn step of SDAVL epistemically grounded rather than reactive. The process model reflects what is actually happening, not what the most recent event suggests might be happening. That distinction is the difference between a governance system that learns and a governance system that overreacts.

4a.3 Predictive Control via LLM-MPC

Classical STAMP control structures are reactive: the controller evaluates the current proposed action against the current process model and issues a control action. This is appropriate for

physical systems where the future trajectory of the controlled process is largely determined by physics and the current action has bounded downstream effects.

Autonomous agent systems are different. An agent action that is individually policy-compliant may be the first step in a trajectory that leads to a hazard state. A sequence of permitted actions may cumulatively produce an outcome that no individual action would have triggered. Reactive control cannot prevent trajectory-level hazards. It can only block individual violations.

LLM-MPC -- the application of classical Model Predictive Control to agent decision-making -- extends ARE's control structure from reactive to predictive. At the Decide step of SDAVL, rather than evaluating only the current proposed action against current state, LLM-MPC simulates forward across a planning horizon: given the current process model, what are the likely future states if this action is permitted? If the predicted trajectory converges on a hazard state within the planning horizon, the coprocessor can block or modify the current action -- before the hazard materializes.

This is qualitatively stronger control in STAMP's terms. STAMP says the controller must issue actions appropriate given the current process model. LLM-MPC says the controller must issue actions appropriate given the anticipated future process model trajectory. A control action that is appropriate for the current state but initiates a hazard trajectory is not, in the extended sense, an appropriate control action.

LLM-MPC is tractable for agent systems in a way it was not for the physical systems STAMP was designed for. The 'plant' in an agent governance context is an LLM operating under a known policy set with a verifiable behavioral history in the Ledger. Forward simulation is computationally feasible within bounded horizons.

Planning horizon and latency tradeoff: production deployment targets a two to three action step horizon. At this horizon, MPC evaluation adds approximately 20--40ms to the Decide step under typical policy complexity -- within the sub-100ms total evaluation budget. Longer horizons increase predictive fidelity but compound prediction error and latency proportionally. The two to three step horizon is the practical bound at which trajectory safety evaluation remains both meaningful and operationally viable.

Prediction accuracy and enforcement semantics: LLM-MPC prediction is approximate, not deterministic. The forward simulation models the most probable action sequence given the current process model and agent behavioral history. When prediction is wrong -- when the agent takes a different path than predicted -- the SDAVL loop corrects at the next Sense step. The process model updates. The next prediction is better informed. Prediction error does not cascade: each action is independently evaluated at enforcement time regardless of what the prior prediction suggested. The predictive layer is advisory in its forward simulation but binding at the enforcement boundary. A predicted hazard trajectory triggers a block verdict. The verdict is enforced. The prediction does not need to be perfect for the enforcement to be correct.

Failure mode: if LLM-MPC evaluation times out or fails, the coprocessor falls back to point-in-time policy evaluation. The control structure degrades gracefully to reactive enforcement. It does not fail open.

Together, evidence-based control and LLM-MPC represent the two directions in which ARE extends STAMP: backward (making past verdicts auditable and epistemically grounded) and forward (making future trajectories predictable and governable). The classical STAMP control loop is closed in both directions.

5. The Agent Passport

STAMP's control structure model contains an implicit identity assumption: the controller knows what entity it is controlling. In physical systems this is satisfied by physical architecture. The controller is wired to specific actuators. The sensor reports from a specific location. Identity is not a runtime concern.

In autonomous agent systems, identity is the hard problem. An agent has no physical presence. Its identity must be constructed, verified, and maintained at runtime. Without a mechanism for this, the control structure cannot attribute actions, cannot enforce per-agent authority, cannot maintain an agent-specific process model.

The Passport is that mechanism.

5.1 What the Passport Contains

A valid Passport encodes: a unique cryptographic identifier for the agent, the issuing principal and full delegation chain, the scope of delegated authority, behavioral constraints inherited from the governance framework, operational boundaries, and an expiry condition. It is signed by the issuing authority and verified by the coprocessor at every verdict.

Passport expiry is a safety mechanism, not an administrative one. An agent operating on a stale Passport is an agent whose authority state may no longer reflect current policy. From STAMP's perspective, this is process model inaccuracy -- the controller's model of the agent's authority does not match the agent's actual authorized scope. ARE treats Passport staleness as a hazard condition. Action is blocked pending re-issuance.

5.2 The Multi-Agent Recursion Problem

A serious objection to any agent governance framework is the recursion question: who governs the governance agent? When agents compose -- when a parent agent delegates to a child agent that may delegate further -- authority propagation becomes a primary attack surface.

Three specific failure modes arise in multi-agent systems that the Passport and Delegation Model are designed to prevent.

Authority amplification through composition: a child agent cannot be granted authority the parent does not hold. The Passport encodes the delegation chain. Every link in the chain is bounded by the issuing agent's own Passport scope. The coprocessor verifies the full chain at verdict time. An agent claiming authority it was not granted produces a chain that does not verify.

Delegation of control logic: an agent cannot delegate the governance function itself. The coprocessor is not an agent. It is architectural infrastructure. An agent cannot instruct the coprocessor to issue verdicts differently on its behalf. The governance layer is not within the scope of what agents can control.

Recursive controller problem: the Posture layer governs the governance system. It monitors the coprocessor, the Ledger write path, and the Passport registry for signs of degradation or

compromise. It does not resolve infinite regress -- no finite system can. It provides one level of meta-governance sufficient for practical safety in production deployments.

What the Passport and Delegation Model do not resolve: emergent authority accumulation across large agent networks operating over extended time horizons. That is identified as future work. Within the scope of first and second-order delegation -- which covers the vast majority of production multi-agent architectures -- the model is complete.

6. The Operations Ledger

STAMP's central safety concern is process model accuracy. The controller acts on its model. If the model is wrong, actions are wrong. Keeping the model accurate requires feedback that returns accurate information about the controlled process to the controller.

For physical systems, that mechanism is sensors. For autonomous agent systems, sensors do not exist in the relevant sense. The agent generates its own activity records. A controlled process that can report on its own behavior can -- in the absence of integrity controls -- report inaccurately.

The Operations Ledger is the answer.

The Ledger is an append-only, cryptographically secured event log. Every agent action, every policy verdict, every state transition, every control intervention is recorded as an immutable hashchain event. It is simultaneously the system's memory, its audit substrate, and the feedback mechanism that keeps the controller's process model current and verifiable.

6.1 The Four Properties

Immutability means the historical process model cannot be corrupted or revised. The Ledger makes accuracy provable rather than asserted. What happened is what the Ledger says happened.

Append-only means the Ledger captures temporal evolution of system state. STAMP's process model needs to reflect how the system arrived at its current state, not just where it is. The Ledger provides that history.

Cryptographic integrity means the feedback loop is tamper-evident. Each event is bound to its predecessor by hash. A falsified record breaks the chain. Manipulation is detectable.

Agent identity attribution means every Ledger event carries a Passport reference. Every recorded action is attributable to a specific, credentialed agent whose authority and history are verifiable. The process model is a record of who did what, under what authority, with what outcome.

6.2 Trust-Minimization, Not Absolute Trust

The Ledger does not claim to be unbreakable. It claims to make breaking detectable. This is the correct framing for a trust-minimized system and it matters for how the security properties should be understood.

A reviewer will correctly identify that the Ledger's integrity guarantees depend on the integrity of the system writing to it. Three specific attack surfaces exist and should be stated explicitly.

Pre-Ledger actions: an agent acting before a Ledger event is written produces a gap in the record. The coprocessor's physical interposition closes this surface: the verdict must be issued and recorded before the action is permitted to execute. The write is atomic with the permission grant.

Side-channel actions: an agent that can act through channels outside the coprocessor's scope can evade governance entirely. ARE's defense is architectural scope enforcement -- the coprocessor must be the only path to consequential action. This is a deployment requirement, not an ARE component. Gaps in scope are gaps in governance.

Partial write failures: a Ledger event that fails to write produces a detectable gap in the hashchain. The Posture layer monitors for chain continuity. A detected gap triggers circuit-breaking. The system fails safe: it stops rather than proceeding on an incomplete process model.

These are not fatal weaknesses. They are the honest account of where the trust boundary sits and what the architecture does when that boundary is stressed. A system that fails safe under known attack conditions is a safety system. A system that claims it cannot be attacked is not.

6.3 Ledger as Feedback

In the SDAVL loop, the Ledger closes the feedback channel. After a verdict is applied and the agent action executes, the outcome is recorded as a verified Ledger event. That event updates the process model. The coprocessor queries the updated model on the next action request.

The feedback loop runs on every action. Not periodically. Not on exception. Every action produces a Ledger event. Every Ledger event updates the process model. The controller always acts on the most recent verified state of the controlled process.

7. The Governance Coprocessor

Every component of ARE serves the control structure. The Passport makes agents legible. The Ledger makes the process model accurate and verifiable. SDAVL closes the feedback channel.

The Governance Coprocessor is what makes the control structure binding.

STAMP distinguishes between a controller that recommends and a controller that enforces. A controller that can be bypassed is not a controller in STAMP's sense. It is documentation. Accidents happen when control actions are not applied. A governance framework that issues verdicts the agent can proceed without is not preventing accidents. It is recording them.

The coprocessor sits between intent and execution. Nothing passes through without a verdict.

7.1 The Verdict Sequence

When an agent proposes an action, the coprocessor intercepts it before execution. It verifies the agent's Passport -- confirming identity, validating authority scope, checking expiry. It queries the Operations Ledger for current process model state: behavioral history, prior verdicts on similar actions, current system conditions relevant to policy evaluation. It evaluates the proposed action against the policy set encoded in the governance framework. It issues one of three verdicts.

Permit: the action proceeds as proposed. **Modify**: the action is altered to bring it within policy before execution. **Block**: execution is prevented. The attempted action, the basis for blocking, and the agent's Passport state at the time of the attempt are recorded to the Ledger.

Every verdict path produces a Ledger event. The process model updates regardless of outcome.

7.2 Physical Interposition

Physical interposition is what separates enforcement from monitoring. A monitoring system observes what happened and reports it. An enforcement system sits in the causal chain and determines what can happen. STAMP requires the latter.

The coprocessor is architecturally interposed. It is not a sidecar that observes agent traffic. It is not a logging layer that records actions after they execute. It is a gate. The agent cannot reach execution without passing through it.

Guardian-Agent is the open-source reference implementation. OPA-based policy evaluation. Passport verification. Ledger integration. Available at github.com/srex-dev/guardian-agent.

8. Failure Walkthrough: The Architecture Under Stress

Abstract claims require concrete demonstration. The following walkthrough traces a real failure scenario through the architecture step by step. It shows not only that the architecture prevents the failure, but exactly which component does what, and what the Ledger records at each step.

8.1 Scenario: Unauthorized Data Access Attempt

An agent operating in a financial services workflow has been issued a Passport with authority scoped to read-only access of customer account summaries. During execution, it proposes an action to retrieve full transaction histories including PII fields -- a scope that exceeds its Passport authorization and would constitute a GLBA violation if executed.

Step 1: Sense

The agent submits the action request to the coprocessor. The SDAVL loop begins. The coprocessor queries the Operations Ledger for the agent's current process model state: its behavioral history, its Passport scope, and any prior verdicts on similar requests. The Ledger returns the agent's history -- no prior scope violations -- and its current Passport.

Step 2: Decide

The coprocessor evaluates the proposed action against policy. The Passport scope is read-only access to account summaries. The proposed action requests full transaction history with PII. The coprocessor evaluates the delta: the requested action exceeds the Passport authorization. Policy verdict: **Block**.

Step 3: Act

The block verdict is enforced. The action does not execute. The agent receives a policy rejection response indicating that the requested scope exceeds its authorization. No PII is accessed. No GLBA boundary is crossed.

Step 4: Verify

The coprocessor writes a Ledger event. The event records: the agent's Passport identifier, the proposed action in full, the verdict (Block), the policy basis for the verdict (scope exceeded), and a cryptographic hash binding this event to the preceding Ledger state. The chain is intact.

Step 5: Learn

The process model is updated. The agent's behavioral record now includes a blocked scope-violation attempt. On subsequent requests, the coprocessor's process model reflects this history. If the agent repeats the attempt, the prior violation is part of the context the coprocessor evaluates. Repeated scope-violation attempts can trigger Posture-layer escalation -- circuit-breaking, human review, or Passport revocation -- depending on policy configuration.

8.2 Escalation: Adaptive Control Under Repeated Violation

The walkthrough above covers a single violation. The architecture's adaptive control behavior -- what happens when the pattern repeats -- demonstrates that ARE is not static enforcement. It is a learning control structure.

On a second scope-violation attempt by the same agent, the coprocessor queries the Ledger and finds the prior blocked attempt in the process model. Policy configuration determines the response threshold. A second violation may produce an elevated block verdict with a Posture-layer flag: the agent's behavioral pattern is now anomalous relative to its Passport scope.

On a third violation, the Posture layer triggers Passport revocation. The revocation is recorded to the Ledger as a control intervention event, cryptographically bound to the preceding chain. The coprocessor updates its Passport registry. All subsequent action requests from this agent produce an immediate block -- not a policy evaluation, but an identity rejection. The agent cannot act until a new Passport is issued through the governed issuance process.

The Ledger now contains the full escalation path: initial violation, block verdict, second violation, elevated block, third violation, revocation event, subsequent rejections. Every step is attributable, timestamped, and cryptographically bound. A regulator, auditor, or incident reviewer can reconstruct the complete sequence from the Ledger alone. The audit trail is not assembled after the fact. It is the control loop's output.

This is the difference between static enforcement and adaptive control. Static enforcement blocks violations. Adaptive control learns from them, adjusts the process model, and tightens the control response proportionally. ARE provides both.

8.3 What This Demonstrates

The failure was prevented at the enforcement boundary, not detected after execution. No monitoring system, no post-hoc audit, no human reviewer was required. The control structure held.

The Ledger record is complete, attributable, and cryptographically bound. It is sufficient for regulatory audit -- it shows what was attempted, when, by which agent, under what authority, and what verdict was issued. The compliance record is a designed output of the control loop, not a side effect.

The agent's future behavior is governed by an updated process model that includes this event. The control loop learned. The next action request encounters a controller that knows this agent has attempted scope violation. The process model is more accurate after the failure than before it. That is the feedback loop working as designed.

9. Lightweight Formalism

The following formalism is intentionally minimal. Its purpose is to make the control structure's properties precise enough to reason about without requiring full formal verification. A complete temporal logic treatment is identified as future work in Section 12.1.

9.1 State, Action, and Control

Let **S** denote the system state, comprising the Ledger state L and the active Passport registry P:

```
S = (L, P)
```

Let **A** denote a proposed agent action. The control function **C** maps a system state and a proposed action to a verdict:

```
C(S, A) -> { permit, modify(A'), block }
```

Where **modify(A')** produces a modified action A' that is within policy. The control function is total: every (S, A) pair produces a verdict. There is no action that bypasses evaluation.

9.2 The Ledger Update Function

Let **E** denote a Ledger event. Each event is a tuple:

```
E = (agent_id, action, verdict, policy_basis, hash(E_prev))
```

The Ledger update function **U** appends a new event to the Ledger, binding it to the previous state:

```
U(L, E) -> L'  where L' = L ++ E  and  E.hash = H(E_prev)
```

Tamper detection: for any modified event E_m in the chain, H(E_m) != H(E_prev) for all subsequent events. The break is detectable at any subsequent verification step.

9.3 The SDAVL Loop as a State Transition System

The SDAVL loop can be expressed as a state transition system over S:

```
Sense:   query(L, agent_id) -> S_current
Decide:  C(S_current, A) -> verdict
Act:     enforce(verdict) -> outcome
Verify:  E = (agent_id, A, verdict, basis, H(L.head))
Learn:   S' = (U(L, E), P)
```

The system state after each loop iteration is S'. The next iteration operates on S'. The loop is closed: every action produces a state update that the next action evaluates against. The process model cannot become stale within a single action sequence because it is updated after every action.

9.4 The Safety Property

The central safety property of the architecture can be stated as:

```
For all A: A executes -> exists verdict v in {permit, modify} such that
C(S, A) = v
```

Equivalently: no action executes without an affirmative policy verdict. This property holds if and only if the coprocessor is architecturally interposed on all paths to execution. The enforcement of this property is a deployment requirement. Its verification is a matter of architectural audit, not runtime checking.

9.5 The Audit-Completeness Invariant

The Ledger's audit-completeness property can be stated as a formal invariant:

```
For all A_e (executed actions): exists E in Ledger such that E.action =
A_e
```

Every action that executes has a corresponding Ledger event. This is not a monitoring property -- it does not say the Ledger is checked after execution. It is an architectural property: the coprocessor writes the Ledger event as part of the verdict sequence, before execution proceeds. Audit completeness is enforced by construction, not by inspection.

9.6 The Evidence-Completeness Invariant

The evidence-based control property can be stated as a formal invariant:

```
For all verdicts v: exists evidence bundle e such that e explains v
under active policy P
```

Every verdict has an evidence bundle that is sufficient to reconstruct the reasoning behind the verdict under the policy set active at the time of issuance. This invariant is stronger than audit completeness. Audit completeness guarantees that actions are recorded. Evidence completeness guarantees that verdicts are explainable -- that a regulator, auditor, or reviewer can reconstruct not just what was decided but why, from the Ledger alone.

The three invariants together -- safety property, audit-completeness, evidence-completeness -- define the system's full governance envelope. No ungoverned execution. No unrecorded

execution. No unexplained verdict. That is the minimum viable control structure for a high-consequence autonomous system.

10. The Complete Architecture

The four components are a unified system. Safety properties emerge from their interaction, not from any individual component. STAMP's control structure is fully instantiated.

STAMP provides the theory. ARE provides the control structure architecture. The Passport makes agents legible to that structure. The Coprocessor makes the structure physically binding. The Ledger makes the process model accurate and verifiable. SDAVL closes the loop.

10.1 The Interaction Sequence

On every agent action: the agent proposes an action. The coprocessor intercepts it. The coprocessor verifies the Passport -- identity confirmed, authority scoped, expiry checked. The coprocessor queries the Ledger -- process model state retrieved, behavioral history assessed. The coprocessor evaluates policy -- verdict issued. The verdict is enforced. The outcome is recorded to the Ledger as a hashchain event. The process model is updated. The loop closes.

The next action request encounters a controller whose process model reflects verified current reality. That is what STAMP requires. That is what this architecture provides.

10.2 Failure Mode Analysis

A control structure that only works when nothing goes wrong is not a safety system. The architecture's behavior under failure conditions follows directly from the STAMP mapping and the formalism in Section 9.

Ungoverned action: prevented by coprocessor physical interposition. The safety property in Section 9.4 holds: no action executes without a verdict. **Incorrect policy verdict**: mitigated by Ledger-informed process model. The process model reflects verified history, not agent-reported state. **Passport staleness**: treated as a hazard condition. Action blocked pending re-issuance. **Ledger integrity failure**: detected by hashchain verification. Partial write failure triggers Posture-layer circuit-breaking. The system fails safe. **Coprocessor bypass**: detectable through Ledger gaps. An agent acting without producing Ledger events produces a chain discontinuity the Posture layer monitors for.

Each failure mode has a named detection or prevention mechanism. The architecture does not assume perfect operation. It assumes failure will occur and provides a response for each failure class.

11. ARE and NeMo Guardrails: A Direct Comparison

NeMo Guardrails is the most developed enforcement-layer governance framework currently available for agent systems. A direct comparison is warranted because the two frameworks are often discussed in the same context. They are not equivalent. The differences are structural, not matters of degree.

Table 2. ARE vs. NeMo Guardrails: Structural Comparison

Dimension	NeMo Guardrails	ARE
Agent Identity	None	Cryptographic Passport, issued and verified
Authority Scope	None	Delegation Model, enforced at verdict time
Process Model	Stateless -- no history	Operations Ledger -- verified behavioral history
Control Type	Output filter, post-reasoning	Interposed controller, pre-execution
Control Loop	Open -- no feedback	Closed -- SDAVL on every action
Feedback Integrity	None	Hashchain -- tampering detectable
Multi-Agent Support	None	Delegation chains, Passport provenance
Audit Substrate	Logging only	Ledger -- immutable, attributable, cryptographic
STAMP Compliance	Partial -- no process model	Complete -- all fifteen primitives addressed

The critical distinction is in rows three and four. NeMo Guardrails is stateless: it evaluates each output independently without reference to behavioral history. ARE's process model -- the Operations Ledger -- means every verdict is informed by what the agent has done across its entire operational history. A first-time scope violation and a tenth repeated scope violation look the same to NeMo Guardrails. To ARE they are categorically different events with different policy responses.

The second critical distinction is interposition versus filtering. NeMo Guardrails operates after the agent has reasoned and produced an output. It filters what comes out. ARE's coprocessor operates before execution -- before the action reaches the system it would affect. Filtering catches outputs. Interposition prevents actions. In a regulated environment where the action is the violation -- accessing PII, initiating an unauthorized transaction -- filtering after the fact is not sufficient. The action must not execute.

NeMo Guardrails does not claim to be a STAMP-compliant control structure. The comparison is not a criticism of its design goals. It is a clarification of what problem each framework solves. NeMo Guardrails constrains model outputs. ARE governs agent actions. These are different problems with different solutions.

12. Limits of ARE

A framework that claims to solve everything solves nothing. The following are the explicit boundaries of what ARE provides and where it does not apply. Stating these limits is not a concession. It is what makes the claims within the limits credible.

12.1 What ARE Does Not Govern

Agent internal reasoning: ARE governs what agents do, not what they think. The path from internal reasoning to proposed action is not observable. ARE's defense is at the action boundary. An agent can reason incorrectly, unsafely, or adversarially -- and ARE will block the actions that result from that reasoning when they violate policy. But ARE does not prevent the reasoning itself.

Policy correctness: ARE enforces policy without exception. It does not evaluate whether the policy is correct. A misconfigured policy enforced with perfect fidelity is still a misconfigured policy. Policy design is outside ARE's scope. Policy enforcement is ARE's entire scope. Policy generation is external to ARE and may be derived from regulatory frameworks, human governance boards, or automated synthesis systems operating on compliance requirements. ARE makes whatever policy exists unbypassable. It does not make policy right.

Emergent collective behavior: ARE addresses individual agent governance and first and second-order delegation chains. Properties that emerge from the interaction of large agent collectives -- behaviors that no individual agent produces but that arise from aggregate action -- are outside the current model. This is an active area of future work.

Physical side channels: ARE assumes the coprocessor is the only path to consequential action. If an agent can act through channels outside the coprocessor's architectural scope, that action is outside ARE's governance boundary. Scope completeness is a deployment requirement that ARE does not enforce internally.

The root of trust: ARE's cryptographic properties depend on the integrity of the key infrastructure underlying Passport issuance and Ledger signing. A compromised root of trust compromises the Passport and Ledger guarantees. ARE assumes the root of trust is secure. Securing it is an infrastructure requirement.

12.2 Where ARE Fails Safe

Within its scope, ARE is designed to fail safe. An agent that cannot obtain a Passport cannot act. A Ledger event that cannot be written halts the action that would produce it. A coprocessor that detects chain discontinuity triggers circuit-breaking. A Passport that has expired blocks its holder until re-issuance.

The failure modes within ARE's scope produce stops, not silent corruptions. The system does not continue operating on an invalid state. It halts, records, and escalates. That is the correct failure mode for a safety-critical control structure.

13. Production Validation

The architecture described in this paper is not proposed. It has been validated in production in a regulated financial services environment operating under SR 11-7, GLBA, ECOA, and FCRA.

The production record establishes three things. First, the architecture is buildable. The components integrate as described. The interfaces are implementable at scale. Second, the control loop operates continuously under production load. Governance does not become a bottleneck. The coprocessor sits in the execution path without degrading system performance to the point of operational failure. Third, the compliance properties the architecture claims to provide are demonstrable. The Ledger produces the audit substrate regulators require. The Passport system produces the attribution record compliance demands. These are not side effects. They are designed outputs of the control structure.

Performance characteristics: in production deployment, the coprocessor operates at sub-100ms median policy evaluation latency under sustained load across 30 concurrent agents and 789 active API endpoints. Ledger write throughput sustains continuous operation across 12 production applications without back-pressure on the action execution path. These figures are representative of OPA-based policy evaluation at production scale with hashchain Ledger writes. They are stated as order-of-magnitude evidence that interposed governance is operationally viable, not as benchmarks for extrapolation to arbitrary scale.

The reference implementation of the Governance Coprocessor is publicly available at github.com/srex-dev/guardian-agent. The architecture is buildable by others. It has been built.

14. Related Work

14.1 STAMP Applications in Software and AI Systems

STAMP has been applied extensively in aerospace, nuclear, medical devices, and transportation. NASA's adoption of STPA for launch vehicle safety analysis and the FAA's incorporation of STAMP-based analysis into avionics guidance represent the framework's deepest engineering adoptions. Leveson's later work increasingly addresses cyber-physical systems where software plays a central role in the control structure.

Abdulkhaleq and Wagner (2015) developed XSTAMPP, establishing that STAMP's control structure model extends beyond purely physical processes. The framework is not inherently physical. It is inherently structural. This is the foundation for its application here.

Applications to AI and machine learning exist but operate at a different level of abstraction. Salay, Queiroz, and Czarnecki (2018) apply STPA to machine learning components in autonomous vehicles, treating the ML subsystem as a controlled element within a physical control architecture. This work does not address the governance of autonomous agents acting independently across high-consequence decision boundaries. No prior work uses STAMP as the theoretical foundation for the governance layer above deployed agent systems.

14.2 Agent Governance Frameworks

The dominant agent governance approaches -- constraint-based, checkpoint-based, and monitoring-based -- share a structural failure when analyzed through STAMP. They are open-loop. They issue constraints or observe behavior but provide no mechanism for the governance system to update its model of what the agent is doing based on observed outcomes. An open-loop governance system cannot maintain process model accuracy. STAMP predicts the failure modes that result.

NeMo Guardrails (NVIDIA, 2023) is the most developed enforcement-layer framework currently available. Table 2 in Section 11 provides the full structural comparison. The critical gap: NeMo Guardrails provides stateless output filtering. ARE provides a stateful, interposed, closed-loop control structure. These are different solutions to different problems.

14.4 AI Safety and Alignment

The alignment and interpretability research community addresses a related but distinct problem: ensuring that AI systems behave in accordance with human values and intentions. Constitutional AI (Bai et al., 2022), RLHF (Christiano et al., 2017), and interpretability work operate at the model layer.

The distinction is this: alignment asks whether the model will choose to behave safely. ARE asks whether the system will permit unsafe behavior regardless of what the model chooses. Safety engineering's answer to this question has been consistent for fifty years: do not rely on the controlled process choosing to stay within safe boundaries. Build a control structure that enforces boundaries regardless. ARE is that control structure.

15. Conclusion

The problem of governing autonomous agents in high-consequence environments has a fifty-year-old theoretical solution. Leveson's STAMP provides a complete account of what a governance system must be -- what components it requires, what properties they must have, and what failure modes result when they are absent or inadequate.

The reason this solution has not been applied to agent governance is not that STAMP is insufficient. It is that no production implementation rigorous enough to demonstrate the mapping existed. The implementation now exists. This paper provides the formal account of the connection.

Agent Responsibility Engineering, the Agent Passport, the Governance Coprocessor, and the Operations Ledger together constitute a STAMP-compliant control structure for autonomous agent systems. The formal correspondence is complete across fifteen mappings within the scope defined in Section 4.1. Thirteen mappings establish ARE as a complete STAMP-compliant control structure. Two ARE-native extensions -- evidence-based control and predictive control via LLM-MPC -- extend STAMP to cover properties that do not exist in classical safety engineering: the requirement that control actions be epistemically verifiable, and the capability to govern by trajectory rather than by point-in-time policy evaluation. Three components extend STAMP to address properties that do not exist in physical systems. The two formal invariants in Section 9 --

the safety property and the audit-completeness invariant -- define the system's minimum viable safety envelope. The limits of the framework are stated explicitly. A framework that knows its boundaries is more trustworthy than one that claims none.

Intelligence never grants authority. Authority is earned through verifiable policy enforcement.

This is the engineering expression of STAMP's safety constraint model applied to autonomous systems. The controller does not trust the controlled process to stay within safe boundaries. The controller enforces boundaries. The Passport defines who is acting. The Coprocessor enforces what they can do. The Ledger records what they did. SDAVL closes the loop. The control structure holds.

Systems deploying autonomous agents without interposed control structures are not operating advanced AI. They are operating unmanaged processes outside established safety engineering practice. The gap is not theoretical. It is not a matter of degree. It is architectural. A system without a controller is not a governed system. It is a system that has not yet had its accident.

15.1 Future Work

Three directions follow directly from this work. First, formal verification of SDAVL's control loop properties using temporal logic would strengthen the theoretical contribution and provide a basis for certification in safety-critical deployment contexts.

Second, the emergent collective behavior problem -- properties arising from large agent collectives that no individual agent produces -- requires extension of the formal model beyond first and second-order delegation chains. This is the most significant open problem in multi-agent governance.

Third, the ALI/ALO/ALA framework -- Agent Level Indicators, Objectives, and Agreements, transposing SRE's SLI/SLO/SLA model into agent governance -- represents a STAMP-native performance model for agent systems. Where this paper addresses what a control structure must be, ALI/ALO/ALA addresses how to measure whether it is working. That is a follow-on paper.

References

Rabanser, S., Kapoor, S., Kirgis, P., Liu, K., Utpala, S., & Narayanan, A. (2026). Towards a Science of AI Agent Reliability. arXiv:2602.16666. Princeton University.

Fernandez, M. (2026). Agent Control Protocol: Admission Control for Agent Actions. arXiv:2603.18829. TraslaIA.

Anon. (2026). Governance Architecture for Autonomous Agent Systems: Runtime Governance for AI Agents: Policies on Paths. arXiv:2603.16586.

Anon. (2026). Cryptographic Runtime Governance for Autonomous AI Systems: The Aegis Architecture for Verifiable Policy Enforcement. arXiv:2603.16938.

Anon. (2025). Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement. arXiv:2508.18765.

Abdulkhaleq, A., & Wagner, S. (2015). XSTAMPP: An eXtensible STAMP Platform as Tool Support for Safety Engineering. STAMP Workshop, MIT.

Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862.

Christiano, P., Leike, J., Brown, T., et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS 2017.

Federal Reserve Board. (2011). SR 11-7: Guidance on Model Risk Management. Board of Governors of the Federal Reserve System.

Kershaw, J. R. M. (2025). Guardian-Agent: Open Source Reference Implementation of the ARE Governance Coprocessor. github.com/srex-dev/guardian-agent.

Kershaw, J. R. M. (2026). Agent Responsibility Engineering: Tenets and Foundational Framework. github.com/srex-dev/AgentReliabilityEngineering.

Leveson, N. (1995). Safeware: System Safety and Computers. Addison-Wesley.

Leveson, N. (2011). Engineering a Safer World: Systems Thinking Applied to Safety. MIT Press.

Leveson, N., & Thomas, J. (2018). STPA Handbook. MIT Partnership for Systems Approaches to Safety and Security.

NIST. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology.

NVIDIA. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. github.com/NVIDIA/NeMo-Guardrails.

Salay, R., Queiroz, R., & Czarnecki, K. (2018). An Analysis of ISO 26262: Using Machine Learning Safely in Automotive Software. SAE Technical Paper.

Thomas, J. (2013). Extending and Automating a Systems-Theoretic Hazard Analysis for Requirements Generation and Analysis. MIT PhD Thesis.